ENTERPRISE ARCHITECTURE BLUEPRINT

# AI-Powered School Academic Analytics System

Unified Data Schema and Implementation Framework via Google Cloud SQL & Google Agent Development Kit (ADK)

**Target Engine:** Google Cloud SQL for PostgreSQL

**AI Layer Platform:** Google Agent Development Kit (ADK)

**Foundation Model:** Gemini 2.5 Flash

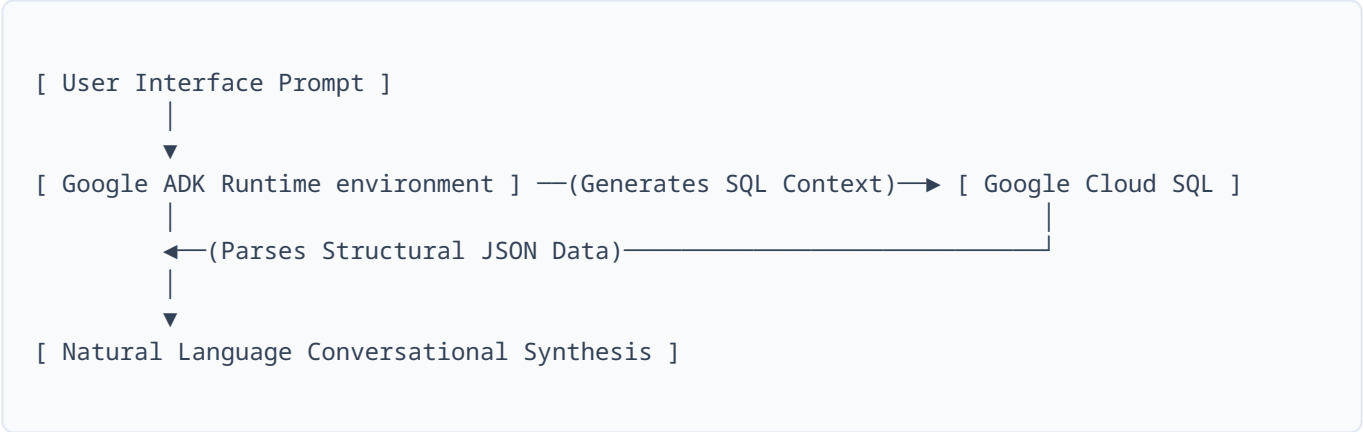
**Document Version:** 1.0.0 (Production Blueprint)

**Date:** July 2026

# 1. Executive Summary & Core Infrastructure Plan

The objective of this project is to implement an autonomous enterprise intelligence layer capable of dynamically executing analytics operations across school student datasets. The framework translates natural language queries directly into performant PostgreSQL code, parses the returned records, and surfaces conversational insights to institutional stakeholders.

To fulfill analytical complexities—specifically comparing dynamic metrics across subjects and applying variable compliance boundaries for student attendance—the architecture relies on **Google Cloud SQL (PostgreSQL)** as the core storage layer and the **Google Agent Development Kit (ADK)** as the runtime processing engine.




# 2. Relational Database Schema Design (DDL)

The relational schema enforces strict structural integrity constraints via primary and foreign key mapping configurations. This enables the agent to evaluate multi-table joins precisely without encountering hallucinations caused by implicit structural ambiguity.

```
-- ========================================================================
-- SYSTEM MIGRATION SCRIPT: ACADEMIC FRAMEWORK SCHEMA
-- TARGET ENGINE: PostgreSQL 14+ / Google Cloud SQL
-- ========================================================================

-- 1. BASE REGISTRATION LOGS
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    grade_level VARCHAR(20) NOT NULL,
    has_medical_accommodation BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. DISCIPLINARY CATALOG
CREATE TABLE subjects (
    subject_id SERIAL PRIMARY KEY,
    subject_name VARCHAR(100) UNIQUE NOT NULL
);

-- 3. QUANTITATIVE PERFORMANCE RECORDS
CREATE TABLE marks (
    mark_id SERIAL PRIMARY KEY,
    student_id INT REFERENCES students(student_id) ON DELETE CASCADE,
    subject_id INT REFERENCES subjects(subject_id) ON DELETE CASCADE,
    exam_type VARCHAR(50) NOT NULL,
    score NUMERIC(5, 2) NOT NULL,
    max_score NUMERIC(5, 2) NOT NULL,
    term VARCHAR(20) NOT NULL,
    CONSTRAINT chk_score CHECK (score <= max_score)
);

-- 4. OPERATIONAL TRACKING REGISTRY
CREATE TABLE attendance (
    attendance_id SERIAL PRIMARY KEY,
    student_id INT REFERENCES students(student_id) ON DELETE CASCADE,
    date DATE NOT NULL,
    status VARCHAR(30) NOT NULL,
    reason TEXT NULL,
    CONSTRAINT chk_status CHECK (status IN ('Present', 'Absent - Unexcused', 'Absent -
Excused'))
);

-- OPTIMIZATION STRATEGY: INDEX GENERATION
CREATE INDEX idx_marks_student_subject ON marks(student_id, subject_id);
CREATE INDEX idx_attendance_student_status ON attendance(student_id, status);
```

## 3. Institutional Analytics Compliance & Reference Queries

The AI runtime translates analytical scenarios into dedicated read-only operational sets. Below are the design blueprints detailing how the system solves specialized rule evaluations natively.

### 3.1 Dynamic Performance Peaks

Evaluating the top performer within a designated subject involves standard join indexing across the performance matrix:

```
SELECT s.first_name, s.last_name, m.score, m.max_score
FROM marks m
JOIN students s ON m.student_id = s.student_id
JOIN subjects sub ON m.subject_id = sub.subject_id
WHERE sub.subject_name = 'Mathematics'
ORDER BY (m.score / m.max_score) DESC
LIMIT 1;
```

### 3.2 Subject Siphon Anomaly Identification

Isolating students demonstrating asymmetrical cognitive capability (e.g., exceptional mathematical capability coupled with systemic weakness across alternative domains) requires standard deviations or explicit relational aggregation boundaries.

```
WITH math_avg AS (
    SELECT student_id, AVG(score / max_score) * 100 as avg_math
    FROM marks m
    JOIN subjects sub ON m.subject_id = sub.subject_id
    WHERE sub.subject_name = 'Mathematics'
    GROUP BY student_id
),
other_avg AS (
    SELECT student_id, AVG(score / max_score) * 100 as avg_others
    FROM marks m
    JOIN subjects sub ON m.subject_id = sub.subject_id
    WHERE sub.subject_name <> 'Mathematics'
    GROUP BY student_id
)
SELECT s.first_name, s.last_name, ma.avg_math, oa.avg_others,
       (ma.avg_math - oa.avg_others) as strength_gap
FROM students s
JOIN math_avg ma ON s.student_id = ma.student_id
JOIN other_avg oa ON s.student_id = oa.student_id
WHERE ma.avg_math >= 85.00 AND oa.avg_others < 70.00
ORDER BY strength_gap DESC;
```

### 3.3 Conditional Medical Attendance Protection

As dictated by project specifications, operational metrics must not apply standard penalties to compliance records containing valid medical exemptions. Thus, the database applies an adjusted attendance equation.

The systemic operational compliance evaluation relies on the following adjusted metric structure:

$$\textit{Attendance Rate} = \Sigma\ (\textit{Present} + \textit{Absent\_Excused}) \div \Sigma\ (\textit{Total Operational School Days})$$

> **CORE GUARDRAIL EXECUTION STRATEGY**
>
> By establishing $\textit{Absent\_Excused}$ as a parameter adding to valid attendance inside the numerator, the data pipeline shields students with dynamic medical profiles from breaching the structural 75% boundary.

```
SELECT
    s.first_name,
    s.last_name,
    COUNT(CASE WHEN a.status IN ('Present', 'Absent - Excused') THEN 1 END) as valid_days,
    COUNT(a.attendance_id) as total_days,
    ROUND((COUNT(CASE WHEN a.status IN ('Present', 'Absent - Excused') THEN 1
END)::NUMERIC /
           COUNT(a.attendance_id)::NUMERIC) * 100, 2) as effective_attendance_rate
```

```
FROM students s
JOIN attendance a ON s.student_id = a.student_id
GROUP BY s.student_id, s.first_name, s.last_name
HAVING (COUNT(CASE WHEN a.status IN ('Present', 'Absent - Excused') THEN 1
END)::NUMERIC /
        COUNT(a.attendance_id)::NUMERIC) < 0.75
ORDER BY effective_attendance_rate ASC;
```

## 4. Unified AI Layer Integration via Google ADK

The following unified code block structures the production wrapper environment. It merges the secure database connection mapping interface with the specialized system runtime agent parameters of the Google Agent Development Kit.

```
# ======================================================================
# UNIFIED COMPONENT RUNTIME ENVIRONMENT FOR GOOGLE ADK
# FILE NAME: unified_analytics_agent.py
# ======================================================================

import os
import psycopg2
from psycopg2.extras import RealDictCursor
from dotenv import load_dotenv
from google.adk.agents.llm_agent import Agent

# Load operational environmental arrays
load_dotenv()

def query_cloud_sql_database(sql_query: str) -> dict:
    """
    Executes an analytical, read-only SQL query against the Cloud SQL PostgreSQL database.
    Use this tool to find information regarding student marks, attendance, specific
subjects,
    and academic profiles.

    Args:
        sql_query (str): A valid PostgreSQL SELECT statement.

    Returns:
        dict: A dictionary containing 'status' and the retrieved dataset 'data' or 'error'
logs.
    """
    # Enterprise Guardrail Execution: Explicit sanitization check
    clean_query = sql_query.strip().upper()
    destructive_terms = ["INSERT", "UPDATE", "DELETE", "DROP", "ALTER", "TRUNCATE"]

    if not clean_query.startswith("SELECT") or any(term in clean_query for term in
destructive_terms):
        return {
            "status": "error",
            "message": "Security Violation: Modifying or non-SELECT expressions are
rejected."
```

```
        }

    try:
        conn = psycopg2.connect(
            host=os.getenv("DB_HOST"),
            database=os.getenv("DB_NAME"),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD"),
            port=os.getenv("DB_PORT", 5432)
        )

        with conn.cursor(cursor_factory=RealDictCursor) as cursor:
            cursor.execute(sql_query)
            result = cursor.fetchall()

        conn.close()
        return {"status": "success", "data": result}

    except Exception as e:
        return {"status": "error", "message": f"Database Execution Failure: {str(e)}"}

# ======================================================================
# GOOGLE ADK AGENT CORE ORCHESTRATION LAYER DEFINITION
# ======================================================================
school_analytics_agent = Agent(
    model='gemini-2.5-flash',
    name='school_academic_analyst',
    description="Engineers dynamic analytical queries against Cloud SQL structures.",
    instruction="""You are an expert institutional data analyst at a school system.
    Your objective is to answer analytical operational queries by mapping intentions to
Postgres SQL statements,
    running them using `query_cloud_sql_database`, and reporting conversational
observations.

    DATABASE MATRIX CONTRACT:
    - Tables available for operational execution: `students`, `subjects`, `marks`,
`attendance`.

    CRITICAL DISCIPLINARY COMPLIANCE INSTRUCTIONS:
    - Attendance metrics use categorization maps: 'Present', 'Absent - Unexcused', or
'Absent - Excused'.
    - If an individual possesses valid medical accommodation credentials or verified
sickness entries,
      the record is classified explicitly as 'Absent - Excused'.
    - DO NOT penalize students for structural absences falling within 'Absent - Excused'.
    - For analytical mapping evaluation, the mathematical baseline is defined as:
      Attendance Rate = Count(Present + Excused) / Count(Total Registered Session Entries)
    - If a student drops below 75% purely via unexcused records, explicitly flag them. Do
NOT flag individuals
      whose records drop below 75% purely due to 'Absent - Excused' tracking blocks.

    CRITICAL ACADEMIC EVALUATION PATHWAYS:
    - The classification 'Strong in math but weak elsewhere' means a student's performance
percentage average
      in 'Mathematics' is significantly superior to their integrated historical average
across all alternate fields.
```

```
    Always run tools to grab explicit record sets prior to compiling structural
outputs.""",
    tools=[query_cloud_sql_database]
)

if __name__ == "__main__":
    print("Google ADK Orchestration Instance initialized successfully.")
    # Production execution checkpoint
    # out = school_analytics_agent.run("Provide compliance outputs on students below 75%
attendance.")
    # print(out.text)
```

## 5. Deployment Execution Plan

| Phase | Execution Requirements |
| --- | --- |
| 1. Data Storage Provisioning | Initialize Cloud SQL PostgreSQL Instance inside GCP. Apply DDL structures provided in Section 2 using standard database interaction tooling. |
| 2. Synthetic Data Scaling | Execute a target generation script using `Faker`, mathematically injecting localized anomaly subsets (e.g., target medical variations, mathematical performance spikes) into your pipeline arrays. |
| 3. Framework Deployment | Map environment configurations ( `.env` credentials) locally or within Google Secret Manager, and initialize the ADK orchestration layer. |